

记忆系统： 让智能体拥有记忆

模仿人类大脑处理信息的方式 —— 编码、存储、检索、整合、遗忘。

主讲人：柠檬酸

人类记忆如何形成

记忆系统的每个技术组件，都映射自一个认知阶段



图 8.3

记忆形成的认知过程



四种记忆类型

WORKING 工作记忆 当前对话的短期记忆，容量受限（默认 50 条），随会话结束清理 纯内存 + TTL	EPISODIC 情景记忆 具体交互事件的长期存储，支持按时间回顾，是"复盘"的基础 SQLite + Qdrant
SEMANTIC 语义记忆 抽象知识、概念与规则——用户偏好、长期指令、领域知识 Neo4j + Qdrant	PERCEPTUAL 感知记忆 图像、音频等多模态信息，支持跨模态检索 模态分离的向量集合

图 8.4

记忆系统的完整工作流程



快速体验记忆功能

先跑起来，再深入原理

```
# 创建具有记忆能力的 Agent
llm = HelloAgentsLLM()
agent = SimpleAgent(name="记忆助手", llm=llm)
memory_tool = MemoryTool(user_id="user123")    # 创建记忆工具

# 添加一条语义记忆 (importance 标记重要程度, 0.0-1.0)
memory_tool.execute("add",
                    content="用户张三是一名Python开发者, 专注于机器学习",
                    memory_type="semantic", importance=0.8)

memory_tool.execute("search", query="前端工程师", limit=3)
```

MemoryTool: 统一入口, 分发处理

自顶向下: 先看统一接口, 再逐个深入关键操作

```
# 一个 execute 分发九种操作
def execute(self, action: str, **kwargs) → str:
    # add / search / summary / stats / update
    # remove / forget / consolidate / clear_all

    if action == "add":    return self._add_memory(**kwargs)
    if action == "search": return self._search_memory(**kwargs)
    # ... 其余操作同理分发
```

add：模拟信息编码

不只是存储内容，更为每条记忆附上丰富的上下文

```
def _add_memory(self, content="", memory_type="working",
                importance=0.5, file_path=None, **metadata)
    # ① 会话 ID 自动管理：每条记忆都有明确的会话归属
    if self.current_session_id is None:
        self.current_session_id = f"session_{datetime.now():%Y%m%d_%H%M%S}"
    # ② 多模态智能处理：自动推断文件类型并保存元数据
    if memory_type == "perceptual" and file_path:
        metadata.setdefault("modality", self._infer_modality(file_path))
    # ③ 上下文自动补充：会话信息 + 时间戳
    metadata.update({"session_id": ..., "timestamp": ...})
```

会话 ID 自动管理

每条记忆都有明确的会话归属，自动区分不同时间段的对话

importance 0.0-1.0

模拟大脑对信息重要性的评估，后续检索、整合、遗忘都依赖它

多模态智能处理

perceptual 类型自动推断文件 modality，并保存相关元数据

完整实现：_add_memory + 四种记忆类型的添加示例

search：记忆系统的核心

在大量记忆中快速找到最相关的内容——语义理解、相关性计算与结果排序

```
def _search_memory(self, query, limit=5, memory_types=None,
                    memory_type=None, min_importance=0.1)
    # 单复数参数标准化：用最自然的方式表达需求
    if memory_type and not memory_types:
        memory_types = [memory_type]
    results = self.memory_manager.retrieve_memories(
        query=query, limit=limit, memory_types=memory_types,
        min_importance=min_importance    # 过滤低质量记忆
    )
    # 输出格式：[类型] 内容预览（重要性：0.80）
```

memory_type / memory_types	min_importance = 0.1	limit
单数复数两种形式都支持，可限定检索的记忆类型	重要性低于阈值的记忆直接过滤，保证结果质量	返回 Top-K 条最相关的记忆，默认 5 条

完整实现：_search_memory（参数标准化 / 格式化输出）+ 调用示例

forget: 选择性遗忘

策略 1

基于重要性

删除重要性低于阈值的记忆

策略 2

基于时间

删除超过指定天数的过时记忆

策略 3

基于容量

接近上限时优先删最不重要的

```
memory_tool.execute("forget", strategy="importance_based", threshold=0.2)
memory_tool.execute("forget", strategy="time_based",          max_age_days=30)
memory_tool.execute("forget", strategy="capacity_based",      threshold=0.3)
```

consolidate: 短期 → 长期

只有真正重要的信息才会被长期保存——系统按重要性阈值自动识别并转换类型



```
memory_tool.execute("consolidate",  
    from_type="working", to_type="episodic",  
    importance_threshold=0.7)
```

MemoryManager：核心协调者



按需启用：memory_types 里有的类型才会初始化对应模块

工作记忆

WORKING MEMORY

最快、最短命的一层——为响应速度而生

定位	存储	检索
存储当前会话的临时上下文，会话结束即清理	纯内存，容量默认 50 条 + TTL（60 分钟）自动过期	TF-IDF 向量化 + 关键词匹配的混合检索，失败时回退关键词
关键机制 · 添加时先做过期清理，容量满时自动移除最低优先级记忆	访问速度极快，代价是重启即丢失—符合"临时信息"的定位	评分 = 语义相似度 × 时间衰减 × 重要性权重

完整实现：WorkingMemory 类（add 过期清理 / retrieve 混合评分）

$$\text{score} = (\text{相似度} \times \text{时间衰减}) \times (0.8 + \text{重要性} \times 0.4)$$

情景记忆

EPISODIC MEMORY

智能体的"经历"，保持事件的完整性与时间序列

定位	存储	检索
长期存储交互事件与学习经历，支持会话级回顾	SQLite 负责结构化数据与复杂查询，Qdrant 负责高效向量检索	结构化预过滤 → 语义向量检索 → 综合评分排序
关键机制 · 维护会话索引 (sessions)，可按会话完整回顾事件序列	混合存储架构 · 事件持久化，重启不丢失	评分同时考虑语义相似与时间近因——既切题、又足够"新鲜"

完整实现：EpisodicMemory 类（会话索引 / 混合检索 / 评分）

$$\text{score} = (\text{向量相似度} \times 0.8 + \text{时间近因} \times 0.2) \times (0.8 + \text{重要性} \times 0.4)$$

语义记忆

SEMANTIC MEMORY

最复杂的一层——知识的结构化表示与关系推理

定位	存储	检索
抽象概念、规则与知识体系，支撑关联推理	Neo4j 知识图谱存实体与关系，Qdrant 向量库做快速相似匹配	向量检索（0.7）+ 图检索（0.3）混合排序
关键机制 · 添加时自动抽取实体与关系，写入知识图谱	文本嵌入采用中文预训练模型，支持中英文 NLP 处理	重要性权重限制在 [0.8, 1.2]，避免盖过相关性

完整实现：SemanticMemory 类（图谱构建 / 向量+图混合排序）

$$\text{score} = (\text{向量} \times 0.7 + \text{图} \times 0.3) \times (0.8 + \text{重要性} \times 0.4)$$

感知记忆

PERCEPTUAL MEMORY

多模态信息的入口——文本、图像、音频统一语义对齐

定位	存储	检索
图像、音频等多模态数据的存储与跨模态检索	按模态分离的向量集合（text / image / audio），避免维度不匹配	同模态精确匹配 + 跨模态语义对齐，时间近因指数衰减
关键机制 · CLIP 编码图像、CLAP 编码音频，统一向量空间语义对齐	生命周期按信息重要性与可用存储空间动态管理	跨模态检索时自动调整评分权重，保证结果多样性

完整实现：PerceptualMemory 类（跨模态检索 / 指数衰减）

$$\text{recency} = e^{-0.1 \times \text{小时数} / 24}$$

（模拟遗忘曲线，最低 0.1）

CLOSING

创建、检索、摘要、遗忘、整合——
一个闭环的智能记忆管理系统，
让 Agent 真正具备类人的记忆能力。